ADR-0001 — Wrapped Update Engine: hawkBit vs Mender vs AOSP-native-only

Field	Value
ADR	ADR-0001
Title	Wrapped update engine: wrap Eclipse hawkBit, wrap Mender, or AOSP-native-only (custom Go rollout engine)
Status	**Proposed**
Revision	2
Fixed (rev 2)	Editorial review fixes: §6 §11.4.74 cite corrected (§13→§15, reuse not testing); §3.1 hawkBit Go-fit reworded "top"→"top-or-tied" (ties `tuf-go-tuf`); §3.4 Option C Go-fit footnoted as flat matrix 3 with "/ Go server" as editorial annotation. No new claims introduced.
Created	2026-06-07
Last modified	2026-06-07
Author	Lead architect (synthesis)
Decision driver	§11.4.8 research-before-implementation; locked decision D2/D3 (native A/B + custom Go control plane; engine wrap target decided by research)
Supersedes	—
Superseded by	—
Related	ADR-0002 (supply-chain trust: TUF/Uptane), ADR-0003 (server topology), ADR-0004 (transport), ADR-0005 (delta updates)
Primary evidence	`ota_landscape_report.md`, `eclipse-hawkbit.md`, `mender.md`, `aosp-update-engine.md`, `commercial-oss-fleet.md`, `additions_synthesis.md`
Anti-bluff rule	Every load-bearing claim cites a source note by slug or a Constitution clause. Items the notes flagged as unverified

Field	Value
	are carried forward verbatim as **UNVERIFIED**. This ADR introduces no facts not present in the underlying notes.

Table of Contents

1. Context

Helix OTA is a universal, deeply decoupled OTA system: a Go server control plane + per-OS client SDKs + dashboard. **Phase 1 targets Android 15** on Orange Pi 5 Max (RK3588 class). [ota_landscape_report]

Two decisions are already **LOCKED** by operator mandate and are *not* re-opened here:

- **D2 (LOCKED):** Device-side update is **native Android A/B** (`update_engine` + AVB/dm-verity + automatic boot-failure rollback), and the server is a **custom, decoupled Go control plane**; an OSS engine is wrapped only where it adds value. [00-master design §2 D2]
- **D3 (LOCKED scope of *this* ADR):** The *wrapped-engine choice* — wrap hawkBit, wrap Mender, or AOSP-native-only — is decided by this evidence-based research ADR, not pre-committed. [00-master design §2 D3; additions_synthesis §7]
- **Mandated stack (LOCKED):** Go + Gin + Brotli + HTTP/3(QUIC)→HTTP/2 fallback, **REST-primary**; **PostgreSQL** relational state + **MinIO/S3** artifact blobs. [ota_landscape_report]

Because D2 fixes the **on-device** mechanism (AOSP native A/B), the question this ADR resolves is narrower than "which whole OTA product." On the device, AOSP `update_engine` is the path regardless of option chosen — it is "the native Android update path; thin wrapper over `applyPayload`" and is adopted, not replaced. [aosp-

update-engine; ota_landscape_report §3.1] The live decision is therefore about the **server-side rollout/campaign engine**: do we *wrap* a third-party orchestration back end (hawkBit or Mender) behind the Go control plane, or *build* the rollout/campaign engine natively in Go and rely solely on AOSP on-device (the "AOSP-native-only" option)?

The two operator-supplied drafts disagreed precisely here: `initial_research.md` proposed wrapping **Mender**; `initial_research_02.md` proposed wrapping **hawkBit**. The reconciliation routed this conflict (C1) to ADR-0001 with **AOSP-native-only as a third option**, noting "the locked strategy biases toward *minimal wrapping* … so a full-platform adopt must clear a high bar." [additions_synthesis §5 C1, §7]

The orchestration capability at stake is Helix's locked staged-rollout design: **percentage phases (5/10/30/…/100) gated by success/error thresholds with pause/halt on breach.** [additions_synthesis §3]

---

2. Decision Drivers & Constraints

1. **Decoupling (§11.4.28).** Each unit has one purpose, a well-defined interface, and is independently testable; the rollout-engine and OS-adapter must remain separate, reusable modules so future OSes/projects can reuse them. [00-master design §4, §11.4.28] Any wrapped engine must sit **behind** the Go control plane via a clean seam and never be publicly exposed.
2. **Minimal-wrap bias (D2).** Wrap OSS only where it adds value; a full-platform adoption must clear a high bar. [00-master design §2 D2; additions_synthesis §5 C1]
3. **Stack conformance.** The control plane and its dependencies must conform to Go + Gin + REST-primary + PostgreSQL + MinIO/S3. [ota_landscape_report]
4. **Android-15 fit.** The chosen engine must not fight native A/B; Android `payload.bin`/slot semantics stay in the Helix client either way. [aosp-update-engine; eclipse-hawkbit §9]
5. **License cleanliness.** No copyleft or proprietary-gated dependency on the orchestration path. [mender §9; eclipse-hawkbit §2]
6. **No-guessing (§11.4.6) / research-first (§11.4.8).** Unverified specifics must be carried as UNVERIFIED and closed before implementation, not assumed. [additions_synthesis §1]

---

3. Options Considered

3.1 Option A — Wrap Eclipse hawkBit

What it is. A mature, domain-independent OTA back end exposing a REST **Management API** (control-plane facing), a polling REST **DDI API** (device facing), and an AMQP **DMF API** (optional federation). EPL-2.0; Spring Boot / Java 21; PostgreSQL-supported; ships as Docker images. Latest release **1.0.3 (2025-04-09)**, 1.0 line declared API-stable / "Eclipse Mature." [eclipse-hawkbit §1, §2]

Evidence for. - **Headline capability is a near 1:1 match to the locked design.** hawkBit provides percentage-based phased rollouts via cascading deployment groups gated by **success (trigger) thresholds** and **error thresholds**, with an **emergency-shutdown** error action and start/pause/resume + approval gate. The locked Helix design (5/10/30/…/100 % phases gated by success/error thresholds with pause/halt) "maps **directly** onto hawkBit rollout groups + trigger/error thresholds + emergency shutdown… a near 1:1 fit and the core argument for wrapping." [eclipse-hawkbit §5; ota_landscape_report §2.3 (staged-rollout 5)] - **Designed to be wrapped.** hawkBit is explicitly "a back end meant to be driven by 3rd-party apps over the Management API," so the Go control plane "becomes that app." Wrap-ability scored 5/5. [eclipse-hawkbit §1, §8; ota_landscape_report §2.3] - **Clean seam, no Android fight.** OS-agnostic; DDI is plain HTTPS poll + token + feedback, trivially implementable on Android; the client bridges DDI→`applyPayload`; Android-specific install logic stays in the Helix client. [eclipse-hawkbit §9] - **Stack-compatible dependencies.** PostgreSQL supported (separate schema/instance recommended); Docker monolith topology fits; **DDI-only avoids RabbitMQ** (AMQP only needed for DMF push). [eclipse-hawkbit §6, §7] - **License.** EPL-2.0, business-friendly. [eclipse-hawkbit §2] - Highest non-device total in the matrix (**36/45**), and top score on staged-rollout and wrap-ability, and top-or-tied on Go-fit (Go-fit 5 ties `tuf-go-tuf`), among control-plane candidates. [ota_landscape_report §2.2]

Evidence against. - **Second runtime/language.** JVM/Spring-Boot/Java-21 is "heavier than the Go control plane; adds a second language/runtime to operate" — the main downside. [eclipse-hawkbit §10; ota_landscape_report §3.2] - **No Android/TUF awareness.** hawkBit treats artifacts as opaque; `payload.bin`/A/B semantics and TUF signing must live in Helix regardless. [eclipse-hawkbit §8, §9] - **UNVERIFIED integration specifics** that must close before coding the Go wrapper: exact Management API rollout **create/start/pause/resume** verbs and create-rollout JSON fields; whether a **management-side event/webhook stream** exists or status must be **polled**; default DDI `pollingTime`; exact DDI `_links`/feedback schema; S3/MinIO artifact-store support matrix in 1.0.x; dynamic-rollout availability. [eclipse-hawkbit §11; ota_landscape_report §5(1)]

3.2 Option B — Wrap Mender

What it is. A mature, production-grade end-to-end OTA solution for **embedded Linux** (and, recently, Zephyr/MCU): Go client (3.x baseline) + microservices server (MongoDB / NATS / Traefik). Open-core licensing. [mender §1, §2, §4, §8, §9]

Evidence for. - Strong robustness/execution model: dual-A/B rootfs commit/rollback, Update Modules abstraction, no-open-ports HTTPS-polling security model — worth studying. [mender §3, §5, §10] - Client is Go (aligns *in principle* with the Go mandate). [mender §8] - High maturity (5/5), widely deployed in embedded Linux/IoT. [mender §1; ota_landscape_report §2.2]

Evidence against (disqualifying). - **No Android support whatsoever.** Official device-support lists Debian-family Linux, Yocto, Buildroot/OpenWRT, Zephyr/MCU — **Android appears nowhere.** A/B is bootloader-driven (U-Boot/GRUB), **not** Android `update_engine`. "Mender cannot be 'pointed at' an Android 15 device… eliminates Mender's wrap value." Android-15 fit and wrap-ability both score lowest (1/5); wrap-ability "Cannot wrap an engine that doesn't run on Android." [mender §7, §10; ota_landscape_report §2.3, §4] - **Needed orchestration is paid/proprietary.** Phased rollouts, dynamic groups, RBAC are **Enterprise-tier**;

scheduled deployments / auto-retry / delta generation are Professional+. The features overlapping Helix's control plane are the **commercially licensed** part; the OSS pieces are client + basic server. Open-core; License scored 2/5. [mender §6, §9; ota_landscape_report §2.3, §4] - **Server stack conflicts with the mandate.** MongoDB / NATS / Traefik microservices vs the locked Gin + PostgreSQL + MinIO/S3 + REST stack. [mender §4, §10; ota_landscape_report §2.3 (Go fit 2)] - Lowest matrix total (**24/45**). [ota_landscape_report §2.2] - **UNVERIFIED:** newest client may be a C++ rewrite (vs verified Go 3.x); exact tier boundaries/pricing; ~70–90% delta savings is a vendor claim. None of these would rehabilitate the option, since the Android gap is decisive. [mender §11]

3.3 Option C — AOSP-native-only (custom Go rollout engine, no orchestration wrap)

What it is. Adopt AOSP `update_engine` on device (as D2 mandates) and **build the rollout/campaign engine natively in Go** rather than wrapping any third-party back end. The Go control plane owns rollout state, cohorts, thresholds, telemetry, and serving.

Evidence for. - **Device side is already covered and is the strongest-scoring path.** `update_engine` provides on-device A/B write to the inactive slot, Virtual A/B COW snapshots, and **automatic bootloader revert on failed boot/dm-verity** — adopted, not replaced. AOSP scores **39/45** (top), with mechanism/rollback/A/B/Android-fit/wrap-ability/license/maturity all 5. [aosp-update-engine; ota_landscape_report §2.3, §3.1] - **Maximally honors the minimal-wrap bias and decoupling.** No JVM, no second runtime; the rollout-engine stays a clean, independently testable Go module (§11.4.28) with no external orchestration dependency. [00-master design §4; ota_landscape_report §3.2] - **Proven patterns exist to copy in Go.** If a wrap is rejected, "build the rollout engine natively in Go and copy Foundries.io's wave state machine (`init→canary→expanding→complete/cancelled`) + device-tag cohorts (group/UUID/percentage targeting) and Memfault's server-side one-click abort." Memfault's AOSP device-client shape (poll-a-releases-HTTP-API → drive UpdateEngine A/B → report result) is the closest existing blueprint. [ota_landscape_report §3.1, §3.2; commercial-oss-fleet] - Full stack conformance by construction (Go + Gin + PostgreSQL + MinIO/S3 + REST).

Evidence against. - **AOSP has no rollout orchestration at all.** Staged-rollout scores 1/5: "None — AOSP applies whatever payload it's handed; all rollout logic is the integrator's." [aosp-update-engine; ota_landscape_report §2.3] Choosing this option means Helix **builds** the threshold-gated cascading rollout engine that hawkBit already provides battle-tested — re-implementing "the hard, well-tested part." [eclipse-hawkbit §8] - Higher build/test burden and schedule risk on the safety-critical rollout-gate path (Constitution §1 floors coverage ≥90% there). [additions_synthesis §5 C8] - **UNVERIFIED:** exact Android-15/RK3588 `update_engine` constants/AIDL headers and Virtual A/B savings (~45% full / ~55% incremental) need board-level confirmation — but these apply to the device side under *every* option. [aosp-update-engine; ota_landscape_report §5(4)]

3.4 Comparison summary

Role-relevant scores from the landscape matrix (read role-specifically, per the report's caveat — low A/B/Android is *expected and not disqualifying* for a control-plane candidate). [ota_landscape_report §2.2, note under §2.2]

Option	Engine slug(s)	Staged-rollout	Go fit	Wrap-ability	License	Android-15 fit	Matrix total
A — Wrap hawkBit	`eclipse-hawkbit` (+ `aosp-update-engine` on device)	**5**	5	**5**	5	4 (via DDI bridge)	36/45
B — Wrap Mender	`mender`	2 (Enterprise-gated)	2	**1**	2	**1** (none)	24/45
C — AOSP-native-only	`aosp-update-engine` (+ custom Go engine)	**1** (build it)	3 (device) / Go server [1]	5	5	**5**	39/45 (device path)

[ota_landscape_report §2.2, §2.3]

Reading: Option B is disqualified on hard facts (no Android; needed orchestration is proprietary; stack conflict). The real trade-off is **A vs C**: wrap hawkBit to *reuse* a near-1:1 rollout engine at the cost of a JVM runtime, versus AOSP-native-only to *build* the rollout engine in Go for a single-runtime, maximally-decoupled stack. The landscape report's own recommendation is to wrap hawkBit **gated** on closing its integration UNVERIFIEDs, with the native-Go engine as the explicit fallback if the team declines the JVM cost. [ota_landscape_report §3.2]

---

4. Decision

Reject Option B (Mender) outright. Adopt Option A (wrap Eclipse hawkBit) as the front-runner for the staged-rollout/campaign engine, GATED on closing the integration UNVERIFIEDs in §5.3; with Option C (AOSP-native-only + custom Go rollout engine) as the pre-approved fallback if any gate fails or the team declines the JVM operational cost. On device, **AOSP `update_engine` (native A/B + Virtual A/B) is adopted and wrapped, not replaced, under every option** — this is fixed by LOCKED decision D2 and is not at issue. [00-master design §2 D2; aosp-update-engine; ota_landscape_report §3.1]

Concretely:

1. **Device side (LOCKED, all options):** Thin Helix Android client around `UpdateEngine.applyPayload`; do not re-implement apply/verify/rollback. [aosp-update-engine; ota_landscape_report §3.1]
2. **Control plane (this decision):** Build the Helix control plane in Go (Gin, REST-primary, Brotli, HTTP/3→HTTP/2, PostgreSQL, MinIO/S3) and **wrap hawkBit** for threshold-gated cascading rollouts + emergency shutdown. hawkBit runs as `hawkbit-monolith` + PostgreSQL + **DDI-only (no RabbitMQ)** + MinIO/S3, **behind** the Go control plane, never publicly exposed; the Go control plane drives it via the Management API. [eclipse-hawkbit §7, §8; ota_landscape_report §3.2]
3. **Fallback (pre-authorized):** If a §5.3 gate fails, build the rollout engine natively in Go, copying Foundries.io's wave state machine + device-tag cohorts and Memfault's one-click abort. This keeps the locked design intact regardless of outcome. [ota_landscape_report §3.2; commercial-oss-fleet]

This honors the LOCKED decisions: native A/B + custom Go control plane (D2), and the wrap target chosen by research (D3), within the mandated stack.

---

5. Consequences

5.1 Positive

- **Reuses battle-tested orchestration.** hawkBit's cascading groups + trigger/error thresholds + emergency shutdown match the locked staged-rollout design near 1:1, avoiding a from-scratch build of the safety-critical rollout-gate. [eclipse-hawkbit §5, §8]
- **Clean decoupling preserved (§11.4.28).** hawkBit sits behind the Management API seam; the rollout concern is isolated and swappable for the Go fallback without touching device or trust layers. [eclipse-hawkbit §8; 00-master design §4]
- **No platform fight / no lock-in.** OS-agnostic engine; EPL-2.0; Android specifics + TUF stay in Helix; Helix's open Go control plane remains the differentiator vs open-client/closed-backend competitors. [eclipse-hawkbit §9, §10; commercial-oss-fleet; ota_landscape_report §4]
- **Stack-aligned dependencies.** PostgreSQL reuse and DDI-only deployment avoid both a stack conflict and the AMQP broker. [eclipse-hawkbit §6, §7]
- **Reversible by design.** The pre-authorized Go fallback means the decision carries no dead-end risk. [ota_landscape_report §3.2]

5.2 Negative

- **Operational cost of a second runtime.** Operating a JVM/Spring-Boot service alongside Go is the principal downside (extra runtime, patching, memory footprint). [eclipse-hawkbit §10; ota_landscape_report §3.2]
- **Helix still owns Android payload + TUF regardless.** hawkBit gives nothing toward `payload.bin` packaging, slot semantics, or supply-chain trust. [eclipse-hawkbit §8, §9]
- **Telemetry mismatch.** DDI feedback states are coarser than `update_engine` progress; Helix needs a separate telemetry channel. [eclipse-hawkbit §9]

- **Decision is conditional.** The wrap cannot be committed to code until §5.3 UNVERIFIEDs close (§11.4.6/§11.4.8); until then the fallback must remain implementable. [eclipse-hawkbit §11; additions_synthesis §1]

5.3 Gating conditions (UNVERIFIED items to close before committing the wrap)

These MUST be confirmed against the live hawkBit 1.0.x reference docs / OpenAPI before the Go wrapper is coded; failure of any pushes the decision to Option C. [eclipse-hawkbit §11; ota_landscape_report §5]

1. **Management API rollout surface** — exact verbs for create/start/pause/resume/approve and create-rollout JSON field names (group count, trigger %, error %). **UNVERIFIED.**
2. **Management-side event push vs. polling** — whether the Go consumer can subscribe to a management event/webhook stream, or must poll for rollout/group status. **UNVERIFIED.**
3. **DDI specifics** — default `pollingTime` and exact `_links`/feedback payload schema (verbatim). **UNVERIFIED** (partially corroborated: `deploymentBase`, `installedBase`, `configData`, `cancelAction`).
4. **Artifact storage backends** — S3/MinIO support matrix in 1.0.x. **UNVERIFIED.**
5. **Dynamic rollouts** and **`DOWNLOADED`-state threshold** configurability in 1.0.x. **UNVERIFIED.**
6. **1.0.3 release date** — releases page shows 2025-04-09; one fetch transcribed 2026; treated as 2025-04-09. **LOW-RISK / confirm.**

Device-side board confirmations (apply under every option, tracked here for completeness): exact Android-15/RK3588 `update_engine` constants/AIDL headers and Virtual A/B savings figures (~45% full / ~55% incremental). **UNVERIFIED.** [aosp-update-engine; ota_landscape_report §5(4)]

Note: **swupdate** was not part of the synthesized findings and is not evaluated here; it is out of scope for the Android Phase-1 engine decision and is carried to the later Linux phase. [ota_landscape_report §4 note, §5(8)]

---

6. Compliance Notes (HelixConstitution)

Clause	How this ADR complies
§11.4.6 (no guessing)	No unverified specific is asserted as fact; every hawkBit integration unknown is carried as **UNVERIFIED** in §5.3 and made a gate before implementation. [additions_synthesis §1; eclipse-hawkbit §11]
§11.4.8 (research before implementation)	This ADR is the evidence-based research decision mandated by D3; it cites the landscape report and stack notes throughout and defers the binding commit until the §5.3 gates close. [00-

Clause	How this ADR complies
	master design §2 D3, §13; additions_synthesis §7]
§11.4.28 (decoupling)	The rollout engine is an isolated, independently testable module behind the Management API seam; hawkBit is swappable for the Go fallback without touching the device or trust layers; OS-adapter and ota-protocol stay separate. [00-master design §4, §13 mapping; eclipse-hawkbit §8]
§11.4.74 (catalogue-first reuse)	Decision favors reusing battle-tested orchestration (hawkBit) and verified Helix submodules over re-implementation, consistent with catalogue-first reuse. The fallback likewise reuses documented Foundries.io/Memfault patterns. [00-master design §10, §15; additions_synthesis §1]
§1 (four-layer testing / coverage)	Whichever path ships, the rollout-gate is safety-critical and floored at ≥90% coverage with mutation immunity per §1, superseding any single percentage target. [additions_synthesis §5 C8]
D2 / D3 (locked decisions honored)	Native A/B + custom Go control plane (D2) is preserved under all options; the wrap target is chosen by research (D3), with hawkBit selected and the AOSP-native-only fallback pre-authorized. [00-master design §2 D2/D3]

7. Status

Proposed. This ADR records the front-runner (wrap hawkBit) and the pre-authorized fallback (AOSP-native-only + custom Go rollout engine), and rejects Mender. It becomes **Accepted** only once the §5.3 gating UNVERIFIEDs are closed against live hawkBit 1.0.x sources (or, on any gate failure, by formally selecting the AOSP-native-only fallback). Trust-layer decisions are deferred to ADR-0002 (TUF/Uptane); server topology to ADR-0003. [eclipse-hawkbit §11; additions_synthesis §7]

---

1. The landscape matrix score for `aosp-update-engine` Go-fit is a flat **3** ("N/A on device; Go fits server side"). The "/ Go server" annotation is editorial, not a second matrix score — it flags that Option C's custom Go control plane is itself fully Go-conformant. [ota_landscape_report §2.2]